

PROJECT DOCUMENT

Implement Client Script & UI Policy (Incident)

ServiceNow – Incident Management

Team ID	SWTID-2026-4373
Team Size	4
Team Leader	MALNI M
Team Member	JAYAPRIYA S
Team Member	Jothika
Team Member	KUZHAIMANGAI G

Prepared as a practical ServiceNow project document for Incident form customization.

1. Project Overview

This project demonstrates how Client Scripts and UI Policies can be used to customize the ServiceNow Incident form. The implementation focuses on improving form behavior, enforcing user-interface rules, and guiding users when incident records are created or updated.

2. Project Objectives

1. Create a Client Script on the Incident table to control form behavior dynamically.
2. Create a UI Policy to apply client-side field behavior based on incident conditions.
3. Improve data entry and user experience on the Incident form.
4. Make important fields mandatory or apply appropriate form controls when required.
5. Test the configuration and verify that the expected behavior works correctly.

3. Technologies / Platform Used

Platform	ServiceNow
Application Area	Incident Management
Table	Incident [incident]
Configuration	Client Script and UI Policy
Execution	Client-side / Form level

4. Client Script Implementation

A Client Script is JavaScript that runs on the client side of a ServiceNow form. It can respond to form events such as loading a record, changing a field, or submitting a record.

4.1 Example: onChange Client Script

The following example demonstrates an onChange Client Script. When the Impact value is changed to High (1), the script displays an informational message and makes the Urgency field mandatory.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {  
    if (isLoading) {  
        return;  
    }  
  
    if (newValue == '1') {  
        g_form.setMandatory('urgency', true);  
        g_form.addInfoMessage('High Impact incident: Please review the Urgency.');
```

```
    } else {  
        g_form.setMandatory('urgency', false);  
    }  
}
```

4.2 Configuration Steps

1. Navigate to All → System Definition → Client Scripts.
2. Click New.

3. Enter a suitable Name, such as “High Impact Incident Control”.
4. Set Table to Incident.
5. Set Type to onChange.
6. Select the Impact field as the Field name.
7. Enter the JavaScript in the Script field.
8. Click Submit or Update.

5. UI Policy Implementation

A UI Policy controls the behavior of fields on a form according to a condition. It can make fields mandatory, visible, or read-only without requiring a server-side script.

5.1 Example UI Policy

Property	Value
Name	High Impact Control
Table	Incident
Condition	Impact is High (1)
On load	Enabled
Reverse if false	Enabled

5.2 UI Policy Actions

Field	Mandatory	Visible	Read-only
Urgency	True	True	False
Additional relevant field (as required)	Configure as required	Configure as required	Configure as required

5.3 Configuration Steps

1. Navigate to All → System UI → UI Policies.
2. Click New.
3. Enter Name as “High Impact Control”.
4. Set Table to Incident.
5. Define the condition: Impact is High (1).
6. Enable On load when the policy should also apply while loading an existing record.
7. Enable Reverse if false so the field returns to its normal state when the condition is no longer true.
8. Under UI Policy Actions, select the required field and configure Mandatory, Visible, and Read-only properties.
9. Click Submit or Update.

6. Client Script vs UI Policy

Feature	Client Script	UI Policy
Purpose	Dynamic form behavior using JavaScript	Control field properties based on conditions
Typical events	onLoad, onChange, onSubmit, onCellEdit	Form load and condition changes
Can run custom logic?	Yes	Limited; mainly field state control
Examples	Messages, calculations, validation	Mandatory, visible, read-only

7. Testing and Validation

Test Case	Expected Result	Status
Open a new Incident	Incident form loads normally	Pass
Set Impact to High	Configured UI Policy / Client Script behavior is triggered	Pass
Check Urgency	Urgency becomes mandatory according to configuration	Pass
Change Impact from High to another value	Reverse condition restores normal field behavior	Pass
Submit the Incident	Record is submitted according to configured validation	Pass

8. Expected Outcome

After successful implementation, the Incident form responds appropriately when the Impact value is changed. The UI Policy controls the selected field properties, while the Client Script can provide dynamic behavior and user guidance. This helps maintain consistent data entry and improves the usability of the Incident form.

9. Conclusion

The project successfully demonstrates the use of ServiceNow Client Scripts and UI Policies for Incident form customization. By combining declarative UI configuration with client-side scripting, the Incident form can provide better validation, clearer user guidance, and more controlled field behavior. These concepts are useful for building maintainable ServiceNow applications.

10. Team Members

Role	Name
Team Leader	MALNI M
Team Member	JAYAPRIYA S
Team Member	Jothika
Team Member	KUZHAIMANGAI G

End of Project Document